The DAG and Graph Rungs: Failure of Thinness and Finiteness

Extracted from a machine-checked development

Abstract

The network directed container of a polytree is thin and has finite position sets whenever the underlying graph is finite and acyclic. Both conclusions fail as soon as one leaves the polytree rung of the structural ladder. We exhibit two elementary counterexamples. First, extending a wine-shipping graph by a Singapore transshipment produces a directed acyclic graph with two distinct routes between the same ports, so the network is not thin. Second, a two-node scrap-metal cycle yields infinitely many positions at a single node (one per number of laps) and a nontrivial endomorphism. The proofs of all stated results have been machine-checked; we record only the mathematical content.

Contents

1 Introduction **1**

2 Preliminaries **2**

3 The DAG rung: two routes via Singapore **2**

4 The graph rung: an infinite scrap loop **3**

5 Discussion **4**

1 Introduction

Directed containers of shipping networks sit on a ladder of structural hypotheses. At the polytree rung, paths between nodes are unique and position sets of finite graphs are finite. Climbing the ladder—first to directed acyclic graphs that are not polytrees, then to graphs with directed cycles—destroys these properties in turn.

The present note records two negative results that mark those rungs.

1. **DAG rung.** On a directed acyclic graph obtained by adding a Singapore transshipment alternative to a direct Melbourne–Thailand sea leg, there are two distinct paths between the same endpoints. Hence network(G) is not thin.
2. **Graph rung.** On a two-node directed cycle modelling a scrap loop between a rolling mill and a melt shop, the set of positions at the rolling mill is infinite: the map sending $n \in \mathbb{N}$ to the path that traverses the cycle n times is injective. The one-lap path is a nontrivial endomorphism (distinct from the root).

The development works throughout in a single universe of types. Proofs are indicated by the phrase “by lean,” meaning they have been checked by a proof assistant rather than written out in prose.

What this note claims. In ordinary mathematical language: acyclicity alone does not imply unique paths, and the presence of a directed cycle yields infinitely many endomorphisms at a node of the free category. Both phenomena are witnessed by fully explicit finite graphs and fully explicit paths.

2 Preliminaries

We work with directed graphs and their path types as in the network development: nodes V, edges E, endpoint maps src and tgt, and inductively defined finite directed paths with concatenation. The network directed container network(G) has shapes V and positions at v given by endpoint–path pairs $\sum_w \mathrm{Path}_G(v, w)$.

Definition 2.1 (Path length)**.** The **length** $|p|$ of a path p is the number of edges it contains: $|\mathrm{nil}(v)| = 0$ and $|\mathrm{cons}(e, p)| = |p| + 1$.

Length is a convenient invariant for distinguishing paths without comparing them edge by edge.

3 The DAG rung: two routes via Singapore

We extend the eleven-facility Sawasdee Cellars shipping graph by a Singapore transshipment option between Melbourne Port and Thai Port.

Definition 3.1 (Extended Sawasdee graph)**.** Let G' be the directed graph whose nodes are the original eleven facilities together with an additional node singapore, and whose edges are the original ten shipping legs together with two new legs

$$\begin{aligned} \mathrm{sing}_1 &: \mathrm{melbournePort} \to \mathrm{singapore}, \\ \mathrm{sing}_2 &: \mathrm{singapore} \to \mathrm{thaiPort}. \end{aligned}$$

The original direct sea leg $\mathrm{sea} : \mathrm{melbournePort} \to \mathrm{thaiPort}$ is retained. The graph G' is acyclic (it remains a DAG) but is no longer a polytree: the two outgoing edges sea and sing_1 from Melbourne Port reconverge at Thai Port.

Definition 3.2 (Two paths)**.** Define paths in G' from melbournePort to thaiPort by

$$\rho_{\mathrm{direct}} = \mathrm{cons}\big(\mathrm{sea},\, \mathrm{nil}(\mathrm{thaiPort})\big), \tag{1}$$

$$\rho_{\mathrm{transship}} = \mathrm{cons}\big(\mathrm{sing}_1,\, \mathrm{cons}\big(\mathrm{sing}_2,\, \mathrm{nil}(\mathrm{thaiPort})\big)\big). \tag{2}$$

Proposition 3.3 (Lengths)**.** *One has* $|\rho_{\mathrm{direct}}| = 1$ *and* $|\rho_{\mathrm{transship}}| = 2$.

Proof. By lean. □

Theorem 3.4 (DAG rung: failure of thinness)**.** *The type* $\mathrm{Path}_{G'}(\mathrm{melbournePort}, \mathrm{thaiPort})$ *is not a subsingleton. In particular,* $\mathrm{network}(G')$ *is not thin at* melbournePort*.*

Proof. By lean. □

Remark 3.5. If the two paths were equal, they would have equal length, contradicting Proposition 3.3. Thus distinct shipping alternatives between the same ports—a commonplace in real logistics—immediately leave the thin rung of the ladder.

4 The graph rung: an infinite scrap loop

Definition 4.1 (Scrap-loop digraph)**.** Let G_{scrap} be the directed graph with two nodes $\{\text{rollingMill}, \text{meltShop}\}$ and two edges

$$\begin{aligned} \text{rm2ms} &: \text{rollingMill} \to \text{meltShop}, \\ \text{ms2rm} &: \text{meltShop} \to \text{rollingMill}. \end{aligned}$$

Proposition 4.2 (Length of a concatenation)**.** *For paths* $p : \text{Path}_{G_{\text{scrap}}}(v, w)$ *and* $q : \text{Path}_{G_{\text{scrap}}}(w, u)$*,*

$$|p +\!\!+ q| \;=\; |p| + |q|.$$

Proof. By lean. □

Definition 4.3 (Laps)**.** Write

$$\lambda \;=\; \text{cons}(\text{rm2ms}, \text{cons}(\text{ms2rm}, \text{nil}(\text{rollingMill}))) \tag{3}$$

for one full lap of the cycle (length 2). Define $\text{laps} : \mathbb{N} \to \text{Path}_{G_{\text{scrap}}}(\text{rollingMill}, \text{rollingMill})$ by

$$\text{laps}(0) \;=\; \text{nil}(\text{rollingMill}), \qquad \text{laps}(n+1) \;=\; \text{laps}(n) +\!\!+ \lambda. \tag{4}$$

Proposition 4.4 (Length of n laps)**.** *For every* $n \in \mathbb{N}$*,*

$$|\text{laps}(n)| \;=\; 2n.$$

Proof. By lean. □

Proposition 4.5 (Injectivity of laps)**.** *The map* laps *is injective: if* $\text{laps}(m) = \text{laps}(n)$*, then* $m = n$*.*

Proof. By lean. □

Definition 4.6 (Infinite type)**.** A type α is **infinite** when there exists an injection $\mathbb{N} \hookrightarrow \alpha$.

Theorem 4.7 (Graph rung: infinite positions)**.** *The position set of* $\text{network}(G_{\text{scrap}})$ *at* rollingMill *is infinite. Concretely, the map*

$$n \;\longmapsto\; \langle \text{rollingMill}, \text{laps}(n) \rangle \tag{5}$$

is an injection $\mathbb{N} \hookrightarrow \sum_w \text{Path}_{G_{\text{scrap}}}(\text{rollingMill}, w)$.

Proof. By lean. □

Definition 4.8 (Nontrivial endomorphism)**.** Let

$$\eta \;=\; \langle \text{rollingMill}, \lambda \rangle \;\in\; \text{Pos}\big(\text{network}(G_{\text{scrap}})\big)(\text{rollingMill}) \tag{6}$$

be the one-lap position at the rolling mill.

Theorem 4.9 (Nontrivial endomorphism)**.** *The position* η *is distinct from the root position at* rollingMill*.*

Proof. By lean. □

Remark 4.10. Length distinguishes them: $|\lambda| = 2$ while the empty path has length 0. In the free category on G_{scrap}, η is a non-identity endomorphism of the rolling-mill object; iterating it produces the infinite family of Theorem 4.7.

5 Discussion

What has been established. Two concrete witnesses separate the rungs of the structural ladder for network directed containers.

- On a DAG that is not a polytree, multiple paths between the same endpoints exist, so thinness fails.
- On a graph with a directed cycle, the position set at a node on the cycle is infinite, and nontrivial endomorphisms exist.

Both conclusions are elementary path-counting facts; their role is to mark sharp boundaries for the positive theorems available on the polytree rung.

Scope. The development does not classify all DAGs or all cyclic graphs, nor does it quantify the growth of position sets beyond the injection from $\mathbb{N}$. The examples are minimal by design.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

Acknowledgements

The mathematical content follows the DAG and graph rungs of the supply-chain development that motivated this note. The presentation is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.

[2] D. Ahman and T. Uustalu. Coalgebraic update lenses. In *MFPS*, volume 308 of *ENTCS*, pages 25–48, 2014.